

Relatório de Auditoria de Segurança - mrleorobot.github.io

Revisão orientada por evidência das cinco classes solicitadas: isolamento de dados, autorização, IDOR, segredos e XSS.

DATA	ESCOPO AUDITADO
30/08/2026	Árvore de trabalho correspondente ao commit publicado 8d87d1c4; código de runtime, scripts de build, configurações, CI, dados estáticos e 10 commits alcançáveis do histórico Git.

Nota metodológica. Cada categoria foi primeiro mapeada para a stack detectada. Categorias sem superfície correspondente foram registradas como não aplicáveis. Somente evidências verificadas no código receberam severidade; condições necessárias para exploração foram mantidas junto ao achado.

VISÃO GERAL

Resumo executivo

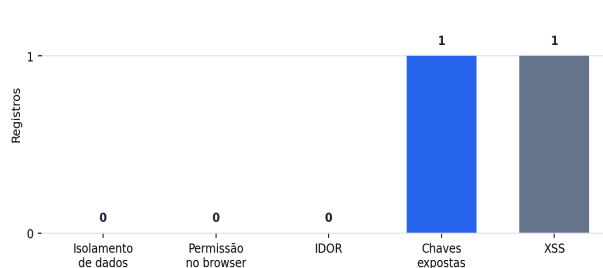
A versão publicada do portfólio apresenta baixa superfície de ataque: é um site estático, sem banco, backend, autenticação, sessão ou rotas de objeto. Não foi confirmada vulnerabilidade crítica, alta ou média. Foram registrados um achado de severidade baixa e uma observação informativa, ambos condicionais e ligados a código residual que não participa do fluxo de produção atual.



Distribuição por severidade



Registros por categoria auditada



Leitura de risco: nenhum segredo real foi encontrado no código atual, nos JavaScript publicados ou nos 10 commits disponíveis localmente. O risco baixo nasce da configuração que passaria uma chave de servidor ao bundle do navegador caso o caminho Vite fosse reativado. A observação de XSS é igualmente latente: a função vulnerável está comentada e o elemento de destino não existe no HTML atual.

FUNDAÇÃO TÉCNICA

Stack detectada e mapeamento

Camada	Tecnologia detectada	Evidência
Linguagem	HTML5, CSS3 e JavaScript ES Modules	index.html; app.js:1-4; package.json:5-16
Frontend	DOM nativo. Não há framework ativo nem bundle de aplicação.	app.js:1-4 importa arquivos JS diretamente; package.json não declara dependências de runtime.
PWA	Service Worker e manifest JSON	sw.js:1-85; manifest.json:1-66
Dados	projects.json estático, transformado por script de build	scripts/sync-projects.mjs:5-7 e 94-117
Backend / API	Não existe no repositório	Nenhum handler HTTP, pasta api/server/routes ou dependencia de servidor.
Banco / ORM	Não existe	Nenhuma dependencia ou configuração Supabase, Prisma, Sequelize, TypeORM, Drizzle, Knex ou banco SQL/NoSQL.
Autenticação	Não existe	Nenhuma sessão, token, login, papel ou gate isAdmin/canEdit.
Build residual	vite.config.ts referencia React/Tailwind e Gemini, mas as dependências e o script build não existem	vite.config.ts:1-13; package.json:7-17; package-lock.json contém apenas o pacote raiz.
Deploy / CI	GitHub Pages e GitHub Actions	.github/workflows/portfólio-qa.yml:1-46. Sem Docker, Helm ou Terraform.

Como as cinco categorias foram adaptadas

Categoria solicitada	Equivalente nesta stack	Resultado
1. Banco sem tranca	RLS, middleware tenant ou filtros user_id/org_id	Não aplicável: não há banco, API ou dados privados por usuario.
2. Permissão no navegador	Gates de papel na UI cruzados com rotas sensíveis	Não aplicável: não há papel, login, escrita privilegiada ou servidor.
3. IDOR	Todos os handlers por ID no backend	Não aplicável: zero handlers de backend.
4. Chaves expostas	Código, config, CI, histórico Git e artefatos cliente	1 achado baixo e nenhum segredo real confirmado.
5. Inputs sem tratamento	29 sinks HTML/eval, fontes externas, URL, localStorage e sanitização	1 observacao informativa; fluxo vulnerável está desativado.

EVIDÊNCIA ANTES DE SEVERIDADE

Cobertura e metodologia

A revisão combinou inventário da árvore, busca estática por padrões, rastreamento manual de fonte até sink, leitura integral dos pontos encontrados e verificação do histórico Git disponível.

- Inventário de código, configurações, dados, CI, arquivos ocultos relevantes e artefatos de deploy.
- Busca por frameworks, ORMs, bancos, middleware, rotas HTTP, gates de papel, IDs e operações privilegiadas.
- Revisão de 29 ocorrências de innerHTML/equivalentes e de todas as atribuições runtime a href/src encontradas.
- Rastreamento das fontes URL, hash, localStorage, sessionStorage, fetch e JSON até os sinks DOM.
- Busca de formatos conhecidos de AWS, Google, GitHub, OpenAI, JWT e chaves privadas em todos os 10 commits alcançáveis.
- Busca por defaults de segredo no formato \${VAR:-valor} e por senhas/JWT/webhooks literais.
- Verificação do bundle efetivo: o deploy atual importa app.js e cinco modulos JavaScript diretamente; não existe dist/build versionado.

Limite do histórico: o clone e raso (shallow) e expoe 10 commits, de 28 a 30 de agosto de 2026. A conclusao historica vale somente para esses objetos Git alcançáveis; uma verificação forense integral exige clone sem profundidade.

Grupo	Arquivos / superfícies revisadas
Runtime web	index.html, offline.html, app.js, script.js, awwwards-upgrade.js, hero-ink.js, evolution.js, sw.js
Build e dados	scripts/sync-projects.mjs, scripts/build-css.mjs, scripts/build-cv.py, take-screenshots.mjs, projects.json
Configuração	package.json, package-lock.json, vite.config.ts, tsconfig.json, manifest.json, .gitignore, .emergent/emergent.yml
QA e deploy	qa/package.json, qa/package-lock.json, qa/playwright.config.mjs, qa/tests/portfólio.spec.mjs, .github/workflows/portfólio-qa.yml
Histórico	10 commits alcançáveis; busca de segredos de alta confiança e defaults inseguros.

POSTURA ATUAL

Pontos fortes e pontos fracos

Pontos fortes

Evidência	O que está correto
Superfície pequena	Não há backend, banco, autenticação, cookies de sessão ou dados privados. As tres primeiras categorias não possuem objeto explorável neste repositório.
Escape no gerador de projetos	scripts/sync-projects.mjs:9-13 define escapeHtml; linhas 62-89 aplicam o escape a textos e atributos gerados.
Overlay protegido	awwwards-upgrade.js:202-218 escapa título, descrição, tags, imagem e link; linhas 269-272 implementam o escape com textContent.
Insercao segura de texto	evolution.js:763-777 e script.js:2048-2057 usam textContent e createElement para conteúdo dinâmico ativo.
Service Worker restrito	sw.js:47-52 ignora metodos diferentes de GET e origens diferentes da origem do site.
CI com menor privilegio	.github/workflows/portfólio-qa.yml:9-10 limita o token a contents: read.
Higiene de segredos	.gitignore:1-7 exclui .env, credentials.json, PEM, chaves e diretorios de credenciais.
Histórico alcançável limpo	Nenhuma credencial de formato conhecido nem default \${VAR:-valor} foi encontrado nos 10 commits disponíveis.

Pontos fracos

- vite.config.ts mantem uma ponte explicita entre GEMINI_API_KEY e o código cliente, embora o pipeline Vite esteja inativo.
- script.js conserva um renderizador desativado que combina dados da API do GitHub/localStorage com innerHTML sem sanitização.
- Não há biblioteca de sanitização nas dependências. Isso não é falha por si só no fluxo atual, mas aumenta o risco de reativar o código residual sem protecao.
- O clone raso impede afirmar que todo o histórico remoto antigo está livre de segredos.

ARQUIVO POR ARQUIVO, LINHA POR LINHA

Achados detalhados

F-01 - Chave de API configurada para injeção em bundle cliente

Severidade	Categoria	Arquivo:linha	Estado
BAIXA	Chaves expostas	vite.config.ts:6-13	Condicional; build Vite inativo

```
6 export default defineConfig(({mode}) => {
7   const env = loadEnv(mode, '.', '');
8   return {
9     base: './',
10    plugins: [react(), tailwindcss()],
11    define: {
12      'process.env.GEMINI_API_KEY': JSON.stringify(env.GEMINI_API_KEY),
13    },
14  }
```

Por que é explorável: o campo define do Vite substitui a expressao no código compilado. Se esse caminho de build for reativado com uma chave real no ambiente, qualquer visitante poderá recuperar o valor no JavaScript ou nas ferramentas de desenvolvimento e usar a cota/permissoes da chave fora do site.

Condição de exploração: exige instalar/reativar Vite, incluir código que referenciae process.env.GEMINI_API_KEY e executar o build com a variavel preenchida. O package.json:7-17 atual não possui script build nem dependências Vite/React/Tailwind; portanto a versão publicada hoje não expoe o valor.

Correção: remover a definicao e o arquivo residual se não forem usados. Se o portfólio voltar a consumir Gemini, encaminhar a chamada por backend/serverless com autenticação, limites e chave somente no servidor. Adicionar teste de CI que falhe quando nomes de segredos forem definidos no bundle cliente.

CONTINUAÇÃO

Achados detalhados

F-02 - Renderizador desativado usa dados externos em innerHTML

Severidade	Categoria	Arquivo:linha	Estado
INFORMATIVA	Inputs sem tratamento (XSS)	script.js:1514-1518, 1595-1617, 1633-1664	Não explorável no deploy atual

```
1514 const cachedData = localStorage.getItem(cacheKey);
1517 const renderRepos = (reposData) => {
1595   const desc = repo.description ? repo.description : "Sem descrição fornecida.";
1601   article.innerHTML = `
1608     <h3>${repo.name}</h3>
1609     <p>${desc}</p>
1613     ${lang}
1615     <a href="${repo.html_url}" ...>Ver Repositório</a>`;
1633   const res = await fetch(`https://api.github.com/users/${username}/repos?...`);
1647   renderRepos(data);
1664 // fetchRecentRepos(); // Desativado: seção substituída pela vitrine estática
```

Por que seria explorável: nome, descrição, linguagem e URL chegam da API/localStorage e entram em um parser HTML sem escape ou sanitização. Se a função for reativada e qualquer campo contiver markup executável, o navegador o interpretará no contexto de mrleorobot.github.io.

Condição de exploração: a chamada está comentada em script.js:1664 e index.html não contém #github-repos-grid. A função retorna antes de ler cache ou rede; não há XSS reproduzível hoje.

Sanitização: não existe DOMPurify ou equivalente. Os fluxos ativos seguros usam textContent/createElement ou escapeHtml manual.

Correção: remover a função morta. Se ela voltar, usar createElement/textContent, aceitar somente URLs https: válidas e aplicar DOMPurify se HTML for indispensável.

REFERENCIA RAPIDA

Tabela consolidada de achados

Severidade	Arquivo:linha	Descrição
BAIXA	vite.config.ts:6-13	Configuração Vite transformaria GEMINI_API_KEY em constante do bundle cliente se o pipeline residual fosse reativado com uma chave real.
INFORMATIVA	script.js:1514-1518, 1595-1617, 1633-1664	Renderizador GitHub desativado interpola resposta externa/localStorage em innerHTML sem sanitização. Não executa na versão atual.

Categorias sem achado

Categoria	Conclusao baseada na stack
Banco sem tranca	Não aplicável. Não há banco, ORM, API, dados privados nem mecanismo de tenant a validar.
Permissão definida no navegador	Não aplicável. Não há autenticação, papéis, UI administrativa nem rotas de escrita.
IDOR	Não aplicável. A busca sistemática encontrou zero handlers de backend e zero operações de objeto por ID.
Chaves hardcoded	Nenhuma chave, token, senha, chave privada ou default público real foi encontrado no código atual ou nos 10 commits alcançáveis.
XSS ativo	Não confirmado. Os sinks ativos recebem constantes/DOM estático ou aplicam escape/textContent. O único fluxo externo sem escape está desativado.

PLANO DE REDUÇÃO DE RISCO

Recomendações priorizadas

P1 Eliminar a ponte de segredo para o cliente

Remover vite.config.ts ou, no mínimo, a definição process.env.GEMINI_API_KEY. Qualquer integração futura com Gemini deve residir em função serverless/backend e aplicar autenticação, limite de uso e rotação da chave.

P2 Apagar ou reescrever o renderizador GitHub morto

Preferir remoção. Se a funcionalidade retornar, usar textContent/createElement e validação estrita de URL https. Cobrir com payloads XSS em teste automatizado.

P3 Adicionar verificação automática de segredos

Executar gitleaks ou ferramenta equivalente no histórico completo em push e pull request. Incluir regra específica que proíba nomes de segredo em Vite define e variáveis VITE_* sensíveis.

P4 Adicionar teste estático de sinks

Falhar o CI quando dados de fetch, localStorage, URL ou hash alcancarem innerHTML sem função de sanitização aprovada. Manter uma lista pequena de exceções justificadas.

P5 Refazer a varredura com clone completo

Executar git fetch --unshallow em ambiente autorizado e repetir a busca de segredos em todos os objetos e branches remotos. Rotacionar imediatamente qualquer credencial encontrada, mesmo removida do HEAD.

Critério de encerramento

O risco residual ficará restrito ao modelo normal de um site estático quando F-01 for removido, F-02 for apagado ou reescrito e o histórico completo receber varredura automatizada. Nenhuma mudança de RLS, autorização ou posse de objeto é necessária enquanto o projeto permanecer sem backend e sem dados privados.

TEXTO COMPLETO EM MARKDOWN

ISSUES PARA O GITHUB

--- ISSUE 1 ---

[Segurança] Remover a injeção de GEMINI_API_KEY no bundle cliente

Labels sugeridas: `security`, `severidade:baixa`

Descrição do problema

O arquivo `vite.config.ts` carrega todas as variáveis de ambiente e define `process.env.GEMINI\_API\_KEY` como uma constante de compilação do Vite. Se esse pipeline residual for reativado com uma chave real, o valor será incorporado ao JavaScript entregue ao navegador.

Isso é explorável porque qualquer visitante consegue baixar o bundle ou inspecionar o código nas ferramentas do navegador e reutilizar a chave fora do portfólio.

Condição atual: o `package.json` não possui script de build nem dependências Vite, React ou Tailwind; portanto o deploy atual não expoe uma chave. A issue previne que a reativacao futura transforme o padrao residual em vazamento real.

Evidência

Arquivo: `vite.config.ts:6-13`

```
``ts
export default defineConfig(({mode}) => {
  const env = loadEnv(mode, '.', '');
  return {
    plugins: [react(), tailwindcss()],
    define: {
      'process.env.GEMINI_API_KEY': JSON.stringify(env.GEMINI_API_KEY),
    },
  };
});
``
```

Impacto

- Consumo indevido da cota da API.
- Custos e bloqueio da chave por abuso.
- Uso das permissões associadas a chave fora do portfólio.
- Necessidade de rotação emergencial caso o build seja reativado sem revisão.

Sugestão de correção

1. Remover `process.env.GEMINI\_API\_KEY` de `define`.
2. Excluir `vite.config.ts` se ele realmente não fizer parte do projeto.
3. Se Gemini voltar a ser usado, criar uma função serverless/backend que guarde a chave somente no servidor e aplique autenticação, limite de uso e monitoramento.
4. Adicionar uma verificação de CI que proíba chaves/nomes de segredo no bundle.

Critérios de aceite

- [] `vite.config.ts` não define nem expoe `GEMINI\_API\_KEY` ao cliente.
- [] Nenhum JavaScript publicado contém o nome ou o valor da chave.
- [] A aplicação continua funcionando sem uma chave Gemini no navegador.
- [] Se existir integração Gemini, a chamada passa por backend/serverless.
- [] O CI falha quando uma chave de servidor é inserida no bundle cliente.

--- FIM ISSUE 1 ---

CONTINUAÇÃO

ISSUES PARA O GITHUB

--- ISSUE 2 ---

[Segurança] Remover ou sanitizar o renderizador GitHub baseado em innerHTML

Labels sugeridas: `security`, `severidade:informativa`

Descrição do problema

`fetchRecentRepos()` lê dados da API do GitHub e do `localStorage` e interpola `repo.name`, `repo.description`, `repo.language` e `repo.html\_url` diretamente em `article.innerHTML`, sem escape, sanitização ou validação de protocolo.

Se a seção for reativada, markup presente nesses campos será interpretado no contexto de `mrleorobot.github.io`, criando uma regressão de DOM XSS.

Condição atual: a chamada está comentada em `script.js:1664` e o elemento `#github-repos-grid` não existe em `index.html`. O fluxo não é explorável no deploy atual; a issue remove o código inseguro antes que ele volte a ser usado.

Evidência

Arquivo: `script.js:1514-1518, 1595-1617, 1633-1664`

```
```\njs\nconst cachedData = localStorage.getItem(cacheKey);\nconst renderRepos = (reposData) => {\n  const desc = repo.description ? repo.description : "Sem descrição fornecida.";\n  article.innerHTML = `\n    <h3>${repo.name}</h3>\n    <p>${desc}</p>\n    <span>${repo.language}</span>\n    <a href="${repo.html_url}">Ver Repositório</a>`;\n};\n\nconst res = await fetch("https://api.github.com/users/.../repos");\nrenderRepos(await res.json());\n\n// fetchRecentRepos(); // Desativado\n```\n
```

## Impacto

- Execução de JavaScript na origem do portfólio caso o fluxo volte a ser ativado.
- Alteração visual, redirecionamento ou phishing dentro da página confiável.
- Persistência temporária de payload no `localStorage` usado pelo renderer.

## Sugestão de correção

Preferência: remover integralmente `fetchRecentRepos()` por ser código morto.

Se a seção voltar:

1. Criar elementos com `document.createElement()`.
2. Inserir nome, descrição e linguagem somente com `textContent`.
3. Validar `repo.html\_url` com `new URL()` e aceitar apenas protocolo `https:`.
4. Evitar atributos de evento inline.
5. Usar DOMPurify imediatamente antes do sink somente se HTML for indispensável.
6. Adicionar teste com payloads como ``.

## Critérios de aceite

- [ ] A função morta foi removida ou não usa `innerHTML` com dados externos.
- [ ] Campos textuais vindos da API/localStorage usam `textContent`.
- [ ] Links aceitam somente URLs `https:` válidas.
- [ ] O payload de teste é exibido como texto e não executa código.
- [ ] Existe teste automatizado cobrindo API e cache manipulados.

--- FIM ISSUE 2 ---